

# Lecture 1

*Module Introduction,  
Java Recap & First Spring Boot App*

BSc Computer Science · Level 5 · 2 Hour Lecture

LO1

LO2

LO5

**Dr Barbara Villarini**



## Learning Outcomes

- ✓ Identify core OOP principles in Java
- ✓ Develop apps integrating Spring Framework
- ✓ Use VCS and professional notation

# Today's Agenda

---



01

## Module Overview & Assessment

What we build, how we're assessed



02

## Java Foundations Recap

Types, methods, control flow — diagnostic review



03

## Classes & Objects — First Contact

The Product class we'll grow all semester



04

## Spring Initializr Walkthrough

Generating, importing, and running a project



05

## Your First REST Endpoint

@RestController, @GetMapping — live demo



06

## Version Control from Day One

Git setup, first commit, push to GitHub



07

## Swagger UI

Making the API visible in a browser



LO1 · LO2 · LO5

# 01

## Module Overview & Assessment Structure

*10 minutes*

# What is 5COSC019W?



## Java

We write everything in Java — the world's most-used enterprise language. You'll become fluent in classes, generics, and the type system.



## Spring Framework

Spring Boot is the industry standard for Java enterprise apps. We introduce Spring from Lecture 1 and build on it each week.



## OOP Principles

Encapsulation, Inheritance, Abstraction, Polymorphism — the four pillars that make software maintainable at scale.

# How You'll Be Assessed

40  
%

## Lab-Based Practical



- › Timed, in-class during seminars
- › Write & test Java code to a specification
- › Use IntelliJ
- › No surprises if you attend every lab
- › Assesses LO1, LO2, LO3
- › Date : During your seminar in Week 9

60  
%

## Enterprise Application



- › Full project — design to deployment
- › Java + Spring Framework
- › 1,000–1,500 word technical report
- › UML, VCS, AI tool reflection
- › Assesses LO1 through LO5
- › Date : 4<sup>th</sup> of January 2027

# The 11-Week Journey

Lectures 1–3

## Phase 1 Foundations

Java recap, Spring Boot,  
classes, collections, service  
layer, UML

Lectures 4–7

## Phase 2 Core OOP

Inheritance, polymorphism,  
abstraction, interfaces,  
Spring IoC & DI

Lectures 8–9

## Phase 3 Quality

Mock Lab Exam, Lab based  
practical, JUnit 5, Mockito,  
MockMvc

Lecture 10-12

## Phase 4 Consolidation

Architecture review, AI  
ethics, sustainability,  
coursework clinic, report  
checklist, peer review

# 02

## Java Foundations Recap

*20 minutes · Diagnostic*



LO1 · LO2

# Why Are We Recapping Java?

## This is DIAGNOSTIC

Not teaching from scratch — we're identifying gaps quickly so the module adapts to the room.



Who has defined a class before?



Who has used a constructor?



Who has written a loop?



Who has used Git?

## What We'll Cover

### Variables & Data Types

int, double, String, boolean

### Methods

Defining, calling, parameters, return types

### Control Flow

if/else, for, while — fundamental logic

### First Class

Blueprint concept — leading into Section 3

# Variables & Data Types

**int****Whole numbers**

```
int quantity = 10;
```

**double****Decimal numbers**

```
double price = 19.99;
```

**String****Text values**

```
String name = "Laptop";
```

**boolean****True or false values**

```
boolean inStock = true;
```



Try it: What do you think Java does if I try to squeeze 19.99 into an int?  
→ Java truncates to 19.

# Methods

## Anatomy of a Method

```
public double calculateTotal(  
    int qty, double unitPrice) {  
    return qty * unitPrice;  
}
```

↑ return type: double — what comes back

↑ parameters: what you pass in

↑ return statement: produces the value

## Calling the Method

```
// Calling calculateTotal  
double total = calculateTotal(10,  
    19.99);  
  
// Total is now 199.90  
System.out.println(total);
```

### Key Rules:

- ✓ Argument types must match parameter types
- ✓ void means the method returns nothing
- ✓ static means you can call it without an object

# Control Flow

## if / else

```
if (price > 100) {  
  
    System.out.println("Expe  
nsive");  
} else {  
  
    System.out.println("Affo  
rdable");  
}
```

Execute different code  
based on a condition

## for loop

```
for (int i = 0; i < 5;  
i++) {  
    System.out.println(  
        "Item " + i);  
}
```

Repeat a block a  
known number of times

## while loop

```
int count = 0;  
while (count < 3) {  
    count++;  
  
    System.out.println(count  
);  
}
```

Repeat until a  
condition becomes false

# Java Warm-Up — Type This Together

```
public class JavaRecap {  
    public static void main(String[] args) {  
  
        // Variables and data types  
        int    quantity  = 10;  
        double price     = 19.99;  
        String name      = "Laptop";  
        boolean inStock  = true;  
  
        // Call a method and store the result  
        double total = calculateTotal(quantity, price);  
        System.out.println(name + " — Total: £" + total);  
    }  
  
    public static double calculateTotal(int qty, double unitPrice) {  
        return qty * unitPrice;  
    }  
}
```

Expected output: Laptop — Total: £199.9

# Java Recap — Key Takeaways



## Types are strict

Java won't silently convert `int` to `String`. Type mismatches are compile-time errors — a feature, not a bug.



## Methods organise code

Group related logic into methods with clear names. A method should do ONE thing.



## Loops = repetition

Any time you find yourself writing the same code twice, a loop is probably the answer.



## Coming next

We're about to take everything above and structure it into a `CLASS` — the fundamental unit of OOP.

# 03

## Classes & Objects

*First Contact*



LO1 · LO2

*20 minutes · The Product class you'll grow all semester*

# The Blueprint Metaphor

## CLASS

### Blueprint

Defines the structure  
shared by all instances

#### Product

```
id : Long  
name : String  
price : double
```



## OBJECTS

### Instances

Each object has its own  
independent state

#### p1

```
id=1  name=Laptop  price=999.99
```

#### p2

```
id=2  name=Headphones  price=49.99
```

# The new Keyword & Constructors

```
// Define the class
public class Product {
    Long id;
    String name;
    double price;

    // Constructor - runs when you write new
    Product(...)
    public Product(Long id, String name, double
price) {
        this.id = id;      // 'this' refers to
THIS object
        this.name = name;
        this.price = price;
    }
}
```

```
// Create two independent objects
Product p1 = new Product(
    1L, "Laptop", 999.99);

Product p2 = new Product(
    2L, "Headphones", 49.99);

System.out.println(p1.name);
// → Laptop
System.out.println(p2.price);
// → 49.99
```

this. refers to the current object. p1.name and p2.name are completely separate values in memory.

# State, Identity & Behaviour



## State

The data an object holds at any moment. Product's state is its current id, name, and price.



## Identity

What makes this object unique. Even two Products with the same name are distinct objects.



## Behaviour

What an object can do — its methods. In the next lecture, Product gets getters, setters, and business logic.

The Product class grows with you all semester



Wk1: fields + constructor

Wk2: private + getters

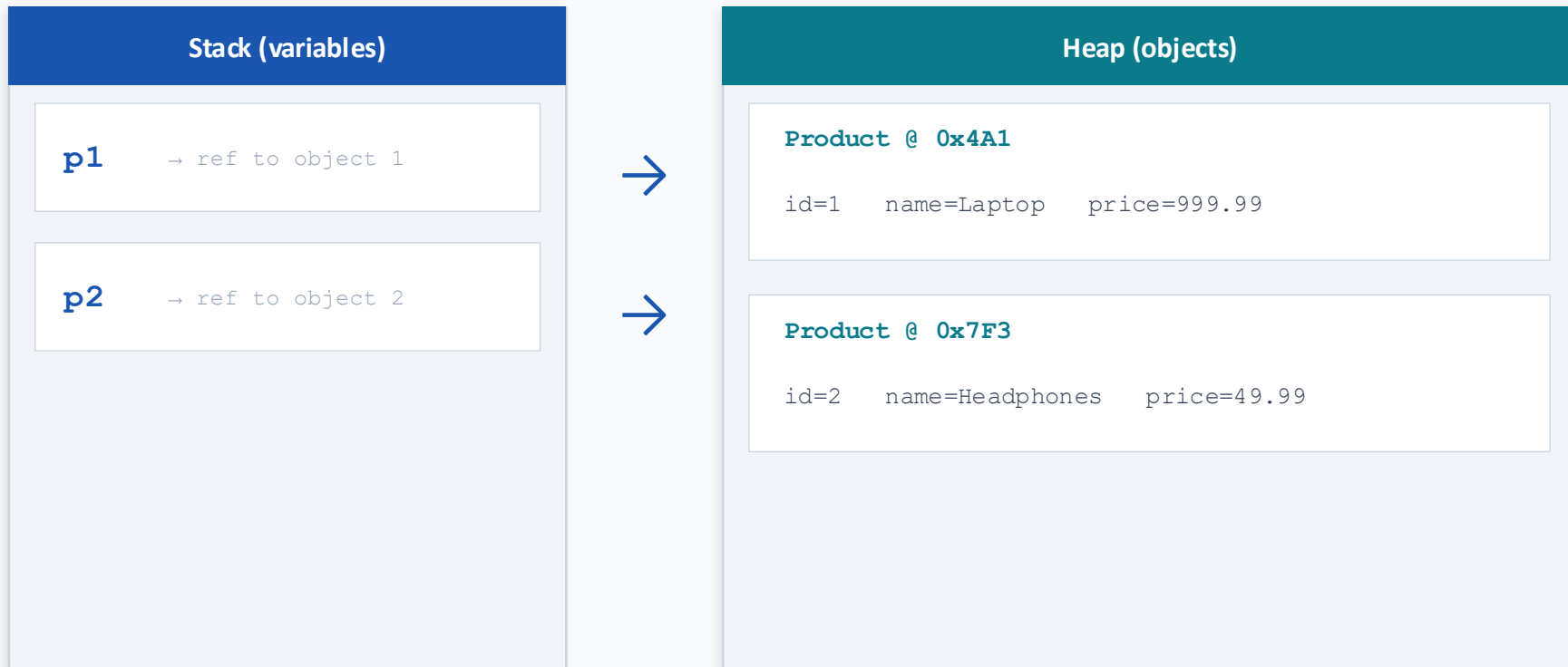
Wk3: service layer

Wk4: inheritance

Wk8: tests

Wk10: Spring repo

# Objects Live in Memory



`p1` and `p2` are references (pointers). Changing `p1`'s name does NOT affect `p2`'s name.

# Quick Check

Q1

**What is the difference between a class and an object?**

→ A class is the blueprint; an object is a specific instance built from it.

Q2

**If I create 100 Product objects, how many Product classes exist?**

→ Just ONE class. All 100 objects share the same blueprint.

Q3

**What does new do?**

→ It calls the constructor and allocates memory on the heap for a new object instance.

# 04

## Spring Initializr Walkthrough

*20 minutes · Live demo — follow along at [start.spring.io](https://start.spring.io)*

# What is Spring Boot?

## Enterprise standard

Used by major banks, Netflix, Amazon — the industry chose Spring for a reason.

## Spring Boot

## Embedded server

Tomcat is built in. No separate install. Run the app like any Java program.

## No XML config

Spring Boot auto-configures almost everything. You write Java, not boilerplate XML.

## Convention over configuration

Sensible defaults out of the box. Override only what you need.

# start.spring.io — Step by Step

01

**Open start.spring.io in the browser**

Everyone open this URL now — follow along.

02

**Project: Maven · Language: Java · Boot: 4.1.1**

Maven manages our dependencies. We'll see pom.xml.

03

**Group: uk.ac.westminster · Artifact: products-api**

This becomes your Java package name.

04

**Packaging: Jar · Java: 25 (depending which version you have installed)**

Jar = self-contained runnable. Java 17 = Long Term Support.

05

**Add dependency: Spring Web**

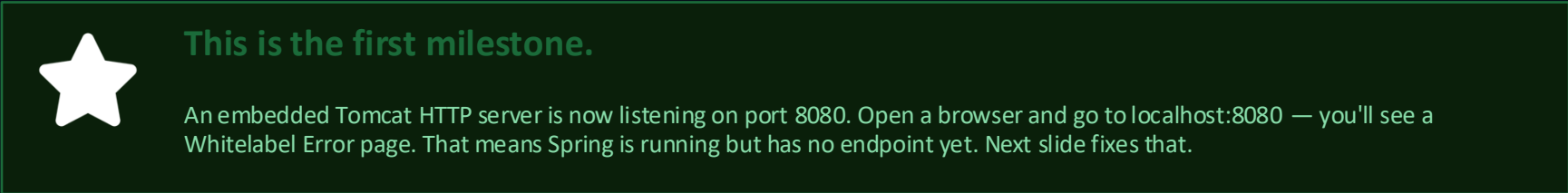
This one dependency adds an HTTP server to your app.

06

**Click Generate → unzip → open in IntelliJ**

File > Open, then wait for Maven to download dependencies. Expand src > main > java > uk.ac.westminster.productsapi. > ProductsApiApplication.java. Double-click to open it.

# You Just Started a Web Server



# This is the first milestone.

An embedded Tomcat HTTP server is now listening on port 8080. Open a browser and go to localhost:8080 — you'll see a Whitelabel Error page. That means Spring is running but has no endpoint yet. Next slide fixes that.

# Understanding pom.xml

```
<!-- pom.xml - your project's dependency manager -->
<project>
  <groupId>uk.ac.westminster</groupId>
  <artifactId>products-api</artifactId>
  <version>0.0.1-SNAPSHOT</version>

  <dependencies>
    <!-- Spring Web = embedded Tomcat + REST support -->
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
  </dependencies>
</project>
```

## groupId / artifactId

Like a unique address for your project on Maven Central.

## dependencies block

Everything inside here gets downloaded and added to your classpath.

## starter-web

A curated bundle: Spring MVC + Jackson JSON + Tomcat server.

## No version needed

Spring Boot manages compatible versions automatically.



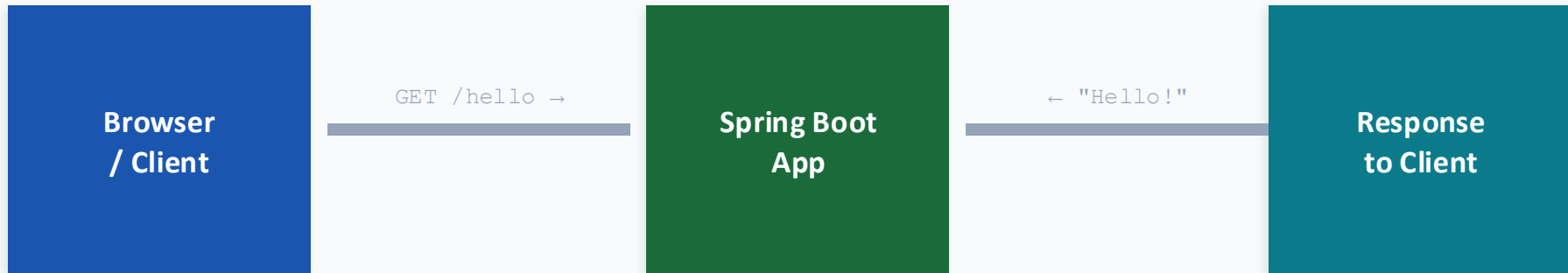
# 05

## Your First REST Endpoint

*20 minutes · @RestController & @GetMapping live*

LO2

# What is a REST Endpoint?



## HTTP Verb

GET reads data. POST creates. PUT updates. DELETE removes.

## Path

/hello is the URL path. Spring matches this to your method.

## Response

Your method's return value becomes the HTTP response body.

## Status

200 OK = success. Spring sets this automatically for you.

# The Key Annotations

## `@RestController`

Marks the class as a Spring component that handles HTTP requests. Spring detects it automatically at startup (component scanning). Every method can return a response body.

## `@GetMapping`

Maps a URL path to a method. `@GetMapping("/hello")` means: when a GET request arrives at `/hello`, run this method. The return value becomes the response.

## `@SpringBootApplication`

On your main class. Activates component scanning, auto-configuration, and the embedded server. The `@RestController` above is discovered because of this.

# HelloController — Type This Now@

```
package uk.ac.westminster.productsapi;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
import java.time.LocalDate;

@RestController
public class HelloController {

    @GetMapping("/hello")
    public String hello() {
        return "Hello from Spring Boot!";
    }

    @GetMapping("/status")
    public String status() {
        return "API running - " + LocalDate.now().toString();
    }
}
```

## What each part does:

### @RestController

Spring finds and manages this class

### @GetMapping

Maps GET /hello → this method

### public String

Method is accessible, returns text

### return ...

This text IS the HTTP response body

# Test It in the Browser

1

Save HelloController.java

2

Stop the app (red square) → Restart (green play)

3

Open browser → navigate to localhost:8080/hello

4

You should see: Hello from Spring Boot!



If I add `@GetMapping("/goodbye")` returning "Goodbye!", what URL will respond to it?

# 07

# Swagger UI

*Making Your API Visible*

15 minutes · One dependency transforms the developer experience



# The Problem — and the Fix

## Without Swagger UI

localhost:8080

### Whitelabel Error Page

Status=404

There was an unexpected error.



You don't know if their API is working



## With Swagger UI

localhost:8080/swagger-ui.html

GET /hello

GET /status

[Try it out] [Execute]



Visual, interactive, auto-generated

# Adding Swagger UI — One Dependency

```
<!-- Add this inside <dependencies> in pom.xml -->
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
  <version>3.1.1</version>
</dependency>

// — That is ALL the code you need. —
// 1. Save pom.xml
// 2. Maven auto-imports the library
// 3. Restart the application
// 4. Open: http://localhost:8080/swagger-ui.html
//
// Every @RestController endpoint appears automatically.
// No extra annotations, no extra configuration.
// Possible troubleshooting: Open the Maven panel on the right edge and click the reload icon
(two circling arrows), then check External Libraries in the project tree for springdoc-openapi-
starter-webmvc-ui and swagger-ui.
```

# What to Look for in Swagger UI



## Endpoint List

Every @GetMapping / @PostMapping endpoint is listed automatically. Add a new endpoint, restart, refresh — it appears.



## Try it out

Click 'Try it out' → 'Execute' to send a real HTTP request to your running app without Postman.



## Response section

See the Response Body, HTTP Status Code (200 OK), and Response Headers. Everything you need to debug.



## Industry standard

Swagger / OpenAPI is used professionally. Learning it now gives you vocabulary for job interviews and industry practice.

# 06

## Version Control from Day One

*15 minutes · Git + GitHub — this is LO5 evidence*



LO5

# Why Version Control from Lecture 1?



## LO5 Requirement

The coursework explicitly requires VCS evidence. A repo with one commit on submission day demonstrates nothing about your development process.



## Professional Habit

In industry, you commit early, commit often, and write meaningful messages. We build that habit from day one.



## Safety Net

Git lets you experiment freely — if you break something, you can always roll back to the last working commit.



## Coursework Evidence

Your commit history tells the story of your development. Assessors can see what you built and when.

# Git in IntelliJ — Six Steps

Steps 1–3 are once per machine. Steps 4–6 are once per project.

**1 Install Git** — check it is there: run `git --version` in the terminal. If you use the university computers launch first Git from AppsAnywhere.

**2 Log in to GitHub** — Settings → Version Control → GitHub → + → Log In with GitHub

**3 Set your identity** — Settings → Version Control → Git → your name and GitHub email. (or through the terminal with the commands: `git config --global user.name "GitHub username"` `git config --global user.email "GitHub_email"`). On github.com go to Settings > Emails. If 'Keep my email address private' is ticked — it is by default — you must use the noreply address shown there as their git email.

**4 Share the project** — VCS in the top menu → Enable VersionControl Integration → Select Git and Enable. Then Git should appear on the top menu. Git menu → GitHub → Share Project on GitHub (untick Private)

**5 Write the first commit** — check the file list, use a real message, click Share

**6 Verify on GitHub** — open the repo in a browser: `src/` and `pom.xml` should be there

# Verify: Files on GitHub

`github.com / [your-username] / products-api`



`src/`

source code



`pom.xml`

Maven configuration



`.gitignore`

tells Git to ignore target/ folder



`README.md`

optional — add one!

**ASSESSMENT:** Submit your GitHub repo URL on Blackboard today. A repo with 11 weeks of commits is compelling evidence for LO5.

# Writing Good Commit Messages

## ✗ Poor Commit Messages

`"fix"`

`"asdf"`

`"done"`

`"changes"`

`"update 3"`

## ✓ Good Commit Messages

`Lecture 1: Spring Boot project created`

`Lecture 2: Product class with encapsulation`

`Lecture 3: CRUD endpoints added to service`

`Lecture 4: Inheritance hierarchy implemented`

`fix: null pointer in getProductById`

*A commit message is a message to your future self. You will thank yourself in Lecture 8.*

# What We Covered Today

|                                                                                    |                              |                                                                                             |
|------------------------------------------------------------------------------------|------------------------------|---------------------------------------------------------------------------------------------|
|   | <b>Module Overview</b>       | Two assessments: Lab (40%, timed) + Enterprise App (60%, full project). 11 weeks, 4 phases. |
|   | <b>Java Recap</b>            | Variables, methods, control flow — the building blocks. We observed where gaps exist.       |
|   | <b>Classes &amp; Objects</b> | A class is a blueprint. new creates instances. Each object has independent state.           |
|   | <b>Spring Initializr</b>     | Generated a Maven project. Added Spring Web. Ran it. 'Started Application' in the console.  |
|   | <b>REST Endpoint</b>         | @RestController + @GetMapping. Tested in the browser. The /hello response is visible.       |
|   | <b>Swagger UI</b>            | One dependency. Interactive API explorer at /swagger-ui.html. Industry standard tooling.    |
|  | <b>Git &amp; GitHub</b>      | Repository created, first commit made, pushed to GitHub. LO5 evidence starts today.         |

# Tutorial 1

## Your tasks in the 2-hour lab:

- 1 Clone the starter project & run the application
- 2 Add Swagger UI dependency to pom.xml
- 3 Add a new field to Person class + second endpoint
- 4 Create a GitHub repo and make your first commit
- 5 Paste your GitHub URL to Blackboard before you leave

## Resources:

[start.spring.io](https://start.spring.io)

Project generator

[springdoc.org](https://springdoc.org)

Swagger dependency docs

[baeldung.com](https://baeldung.com)

Spring Boot tutorials

[git-scm.com/doc](https://git-scm.com/doc)

Git documentation

[Blackboard](#)

Starter project link + submission

## Lecture 2 Preview:

Encapsulation — we'll break the API on purpose to show why private fields matter in Spring Boot.